Projeto - Cidades ESG Inteligentes

HealthTrack API: Prontuário Eletrônico Unificado
com pipeline CI/CD, containers e deploy na Azure

Documentação Técnica
Pipeline CI/CD, Containerização, Orquestração e Deploy em Nuvem

Integrante
Natalia Santana Silva - RM557347

FIAP - 2TDS
Fase: Navegando pelo mundo DevOps

Recurso	Endereço
Repositório	https://github.com/nataliassilva24/cidades-esg-inteligentes
Staging	https://esg-api-9f1e-staging.azurewebsites.net
Produção	https://esg-api-9f1e-prod.azurewebsites.net

1. Visão geral do projeto

A **HealthTrack API** é a API REST desenvolvida na Fase 3 (Java Advanced com Spring Boot) para unificar o prontuário eletrônico do paciente. Nesta fase ela passou a rodar em container, com um pipeline que compila, testa, publica a imagem e faz o deploy em dois ambientes na Azure (staging e produção).

Contexto ESG

Dimensão	Aplicação
Social (S)	Acesso à saúde e redução de desperdício no SUS (cerca de R$ 30 bi/ano em exames repetidos). O endpoint /indicadores/desperdicio calcula os exames repetidos evitados e a economia estimada.
Governança (G)	Privacidade de dados (LGPD): autenticação JWT, acesso por perfil (ADMIN, MEDICO, PACIENTE) e nenhuma credencial versionada no código.

Adaptações em relação ao projeto original

Antes (Fase 3)	Depois (DevOps)	Motivo
Oracle FIAP (um único schema compartilhado)	MySQL 8 (container local e Azure Database for MySQL)	Bancos separados para staging e produção e ambiente local completo
Migrations Flyway em SQL Oracle (sequences)	Migrations em SQL MySQL e entidades com IDENTITY	Compatibilidade com o novo banco
Compose apenas com a aplicação	Compose com aplicação + banco, volume, rede e health checks	Orquestração completa do ambiente
Credenciais e chave JWT com valores padrão no código	Somente variáveis de ambiente e secrets	Segurança
Sem pipeline	GitHub Actions com CI, CD, staging e produção com aprovação	Automação do ciclo de vida
-	Endpoint público / com ambiente e versão	Rastreabilidade do que está em cada ambiente

2. Descrição do pipeline CI/CD

Ferramenta: GitHub Actions, com os workflows versionados em `.github/workflows/`. As imagens vão para o GitHub Container Registry (GHCR) e o deploy é feito no Azure App Service (Web App for Containers). Staging e produção são GitHub Environments, cada um com as suas variáveis e o secret de publicação do seu Web App.

Lógica de branches (Git Flow simplificado)

```
feature/*  --PR-->  develop  --PR/merge-->  main
    |                  |                     |
    CI             CD -> STAGING       CD -> PRODUÇÃO (aprovação manual)
```

Workflow ci.yml - Integração Contínua

Roda em push nas branches feature/** e em todo Pull Request para develop ou main. O merge só é feito com o check verde.

Etapa	Ação
1. Checkout	actions/checkout@v4
2. Setup Java	actions/setup-java@v4 - Temurin 21 com cache do Maven
3. Build + testes	mvn clean verify - 27 testes automatizados (JUnit 5, MockMvc, Mockito, Spring Security Test)
4. Relatório	Upload do relatório Surefire como artefato da execução

Workflow cd.yml - Entrega e Implantação Contínua

Roda em push nas branches develop e main. Os jobs são encadeados com `needs`: o deploy só acontece se os testes e o build passarem.

Job	O que faz	Quando
test	Build e testes automatizados	develop e main
build-image	Build da imagem Docker (Buildx com cache) e push para o GHCR com as tags <sha-do-commit> e staging/production	develop e main
deploy-staging	azure/webapps-deploy@v3 no Web App de staging + smoke test em /actuator/health	somente develop
deploy-production	Aguarda aprovação manual (environment production com required reviewers), faz o deploy no Web App de produção + smoke test	somente main

Nada sensível fica no código. O publish profile de cada Web App é um secret do environment correspondente, e a URL e o nome do app são variáveis do environment. As credenciais do banco e a chave JWT (uma por ambiente) ficam nas App Settings da Azure e chegam ao container como variáveis de ambiente. O Flyway cria e versiona o schema do banco quando a aplicação sobe.

3. Docker: arquitetura, comandos e imagem

Arquitetura

```
AMBIENTE LOCAL (docker-compose.yml)
+---------------------- rede healthtrack-net (bridge) ------------------------+
| [api] healthtrack-api:local  --JDBC (db:3306)-->  [db] mysql:8.0            |
|   porta 8080 publicada                              volume mysql_data       |
|   health: /actuator/health                          health: mysqladmin ping |
+-----------------------------------------------------------------------------+

NUVEM (Azure - Chile Central)
GitHub Actions --push--> GHCR (ghcr.io/nataliassilva24/cidades-esg-api:<sha>)
      |
      +--deploy--> Web App esg-api-9f1e-staging --> MySQL Flexible: healthtrack_staging
      +--deploy--> Web App esg-api-9f1e-prod    --> MySQL Flexible: healthtrack_prod
                   (App Service Plan Linux B1 compartilhado)
```

Dockerfile (multi-stage)

```
# ============================================================
# Etapa 1 - BUILD: compila o projeto com Maven + JDK 21
# ============================================================
FROM maven:3.9-eclipse-temurin-21 AS build

WORKDIR /opt/app

# Copia primeiro só o pom.xml para aproveitar o cache de camadas:
# as dependências só são baixadas de novo quando o pom.xml muda.
COPY pom.xml .
RUN mvn -B -q dependency:go-offline

# Copia o código-fonte e gera o JAR (os testes já rodam no pipeline de CI)
COPY src ./src
RUN mvn -B -q clean package -DskipTests

# ============================================================
# Etapa 2 - RUNTIME: imagem leve só com o JRE 21 (Alpine)
# ============================================================
FROM eclipse-temurin:21-jre-alpine

WORKDIR /opt/app

# Usuário sem privilégios de root (boa prática de segurança)
RUN addgroup -S esg && adduser -S esg -G esg

# Copia apenas o JAR gerado na etapa de build
COPY --from=build /opt/app/target/app.jar app.jar
RUN chown esg:esg app.jar
USER esg

# Versão gravada na imagem pelo pipeline (SHA do commit)
ARG APP_VERSION=dev

# Valores padrão (sobrescritos pelo docker-compose / ambiente de deploy)
ENV APP_ENV=local \
    APP_VERSION=${APP_VERSION} \
    JAVA_OPTS="-XX:MaxRAMPercentage=75.0"

EXPOSE 8080

# O próprio container informa se a aplicação está saudável
HEALTHCHECK --interval=15s --timeout=5s --start-period=40s --retries=5 \
  CMD wget -qO- http://localhost:8080/actuator/health || exit 1

# "sh -c" permite interpolar JAVA_OPTS em tempo de execução
ENTRYPOINT ["sh", "-c", "java $JAVA_OPTS -jar app.jar"]
```

Estratégias adotadas

Estratégia	Benefício
Multi-stage build	A imagem final contém só o JRE 21 Alpine e o JAR, sem Maven nem código-fonte
Cópia do pom.xml antes do código	Cache das dependências: só são baixadas de novo quando o pom.xml muda
Usuário não-root	Se houver uma falha de segurança na aplicação, o processo não tem permissão de root
Configuração por variáveis de ambiente	A mesma imagem roda em local, staging e produção
HEALTHCHECK	Docker, Compose e Azure sabem quando a aplicação está pronta
ARG APP_VERSION	Cada imagem carrega o SHA do commit, exposto na rota /

Orquestração com Docker Compose

Recurso	Configuração
Serviços	api (build do Dockerfile) e db (mysql:8.0)
Volume	mysql_data: persiste os dados do banco entre reinicializações
Rede	healthtrack-net (bridge): a API acessa o banco pelo nome db; o MySQL não é exposto
Variáveis de ambiente	Lidas do arquivo .env (modelo versionado em .env.example)
Dependência	depends_on com condition: service_healthy
Reinício	restart: unless-stopped

Comandos utilizados

```
# Ambiente local
cp .env.example .env
docker compose up -d --build        # build da imagem e subida de API + MySQL
docker compose ps                   # status (healthy)
docker compose logs -f api          # logs da aplicação
docker compose down                 # para (dados preservados no volume)

# Imagem
docker build -t healthtrack-api:local .
docker images healthtrack-api

# Infraestrutura Azure (Azure CLI)
az login
GITHUB_USER=nataliassilva24 ./infra/azure-setup.sh
az group delete -n rg-cidades-esg   # remoção ao final
```

Imagem criada

Item	Valor
Registry	GitHub Container Registry
Nome	ghcr.io/nataliassilva24/cidades-esg-api
Tags	SHA do commit (ex.: abaa67c... em produção, 61dd45f... em staging), staging e production
Base de execução	eclipse-temurin:21-jre-alpine (estágio de build: maven:3.9-eclipse-temurin-21)
Tamanho	cerca de 520 MB conforme o Docker Desktop (a imagem de build com Maven tem 841 MB)

4. Pipeline em execução (build, testes e deploy)

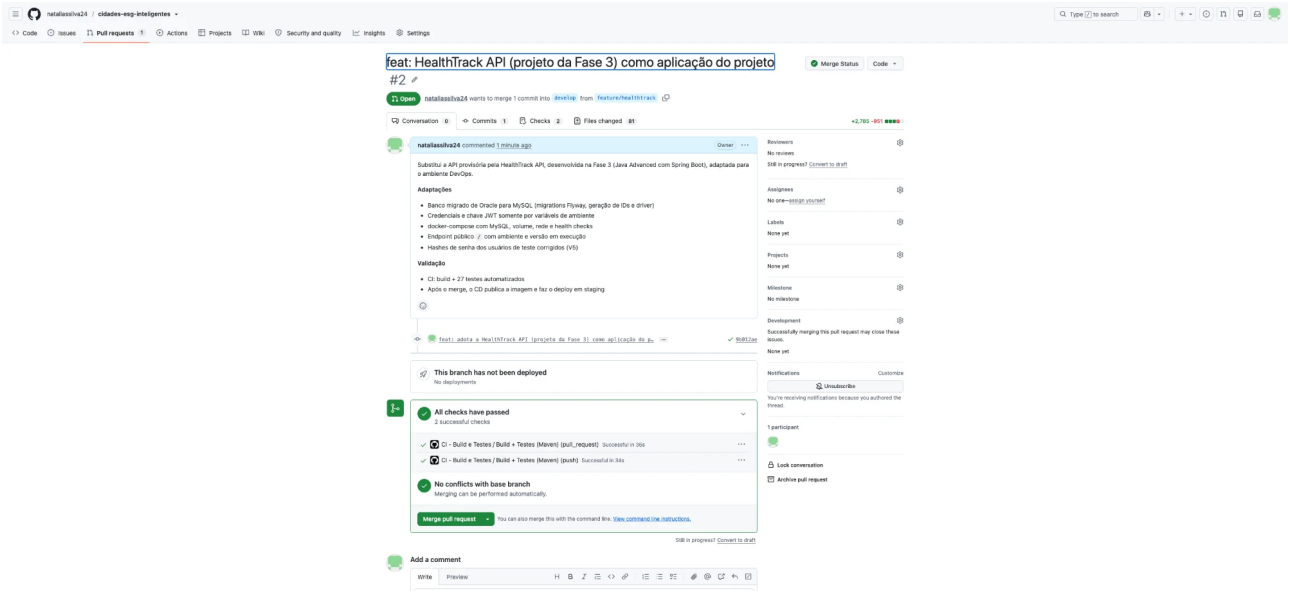


Figura 1 - CI: build e 27 testes automatizados executados no Pull Request feature/healthtrack -> develop

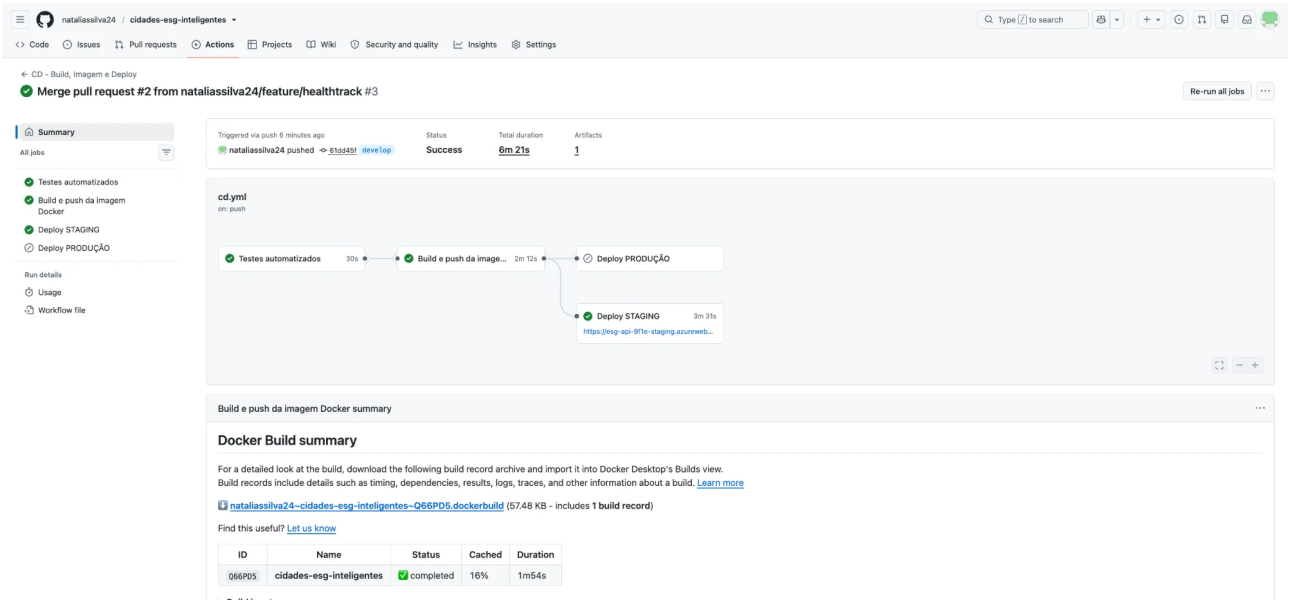


Figura 2 - CD na branch develop: testes, build/push da imagem e deploy em staging (produção ignorada)

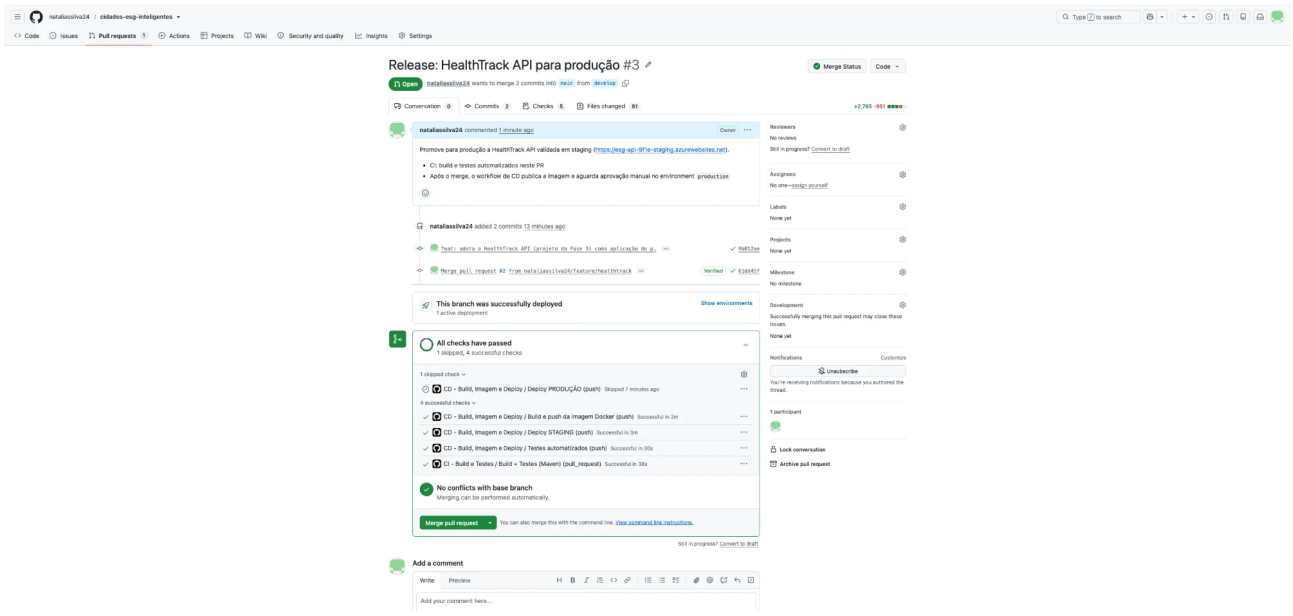


Figura 3 - Pull Request de release develop -> main, com staging já implantado e todos os checks verdes

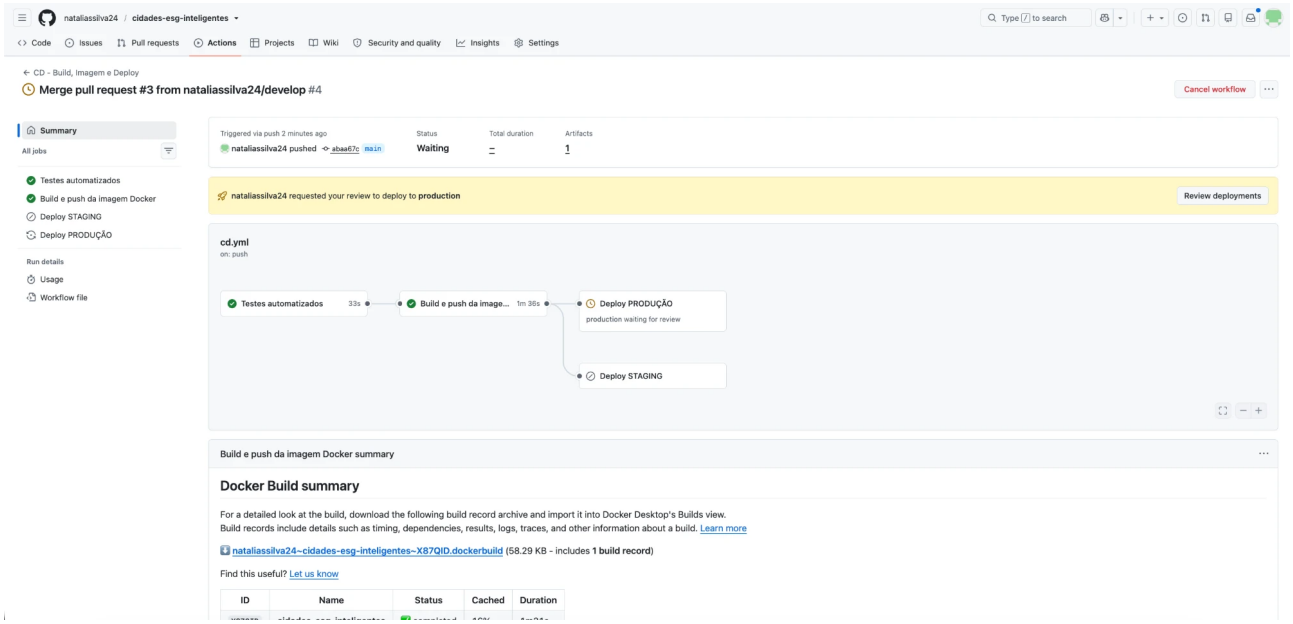


Figura 4 - CD na main: deploy de produção aguardando aprovação manual do environment production

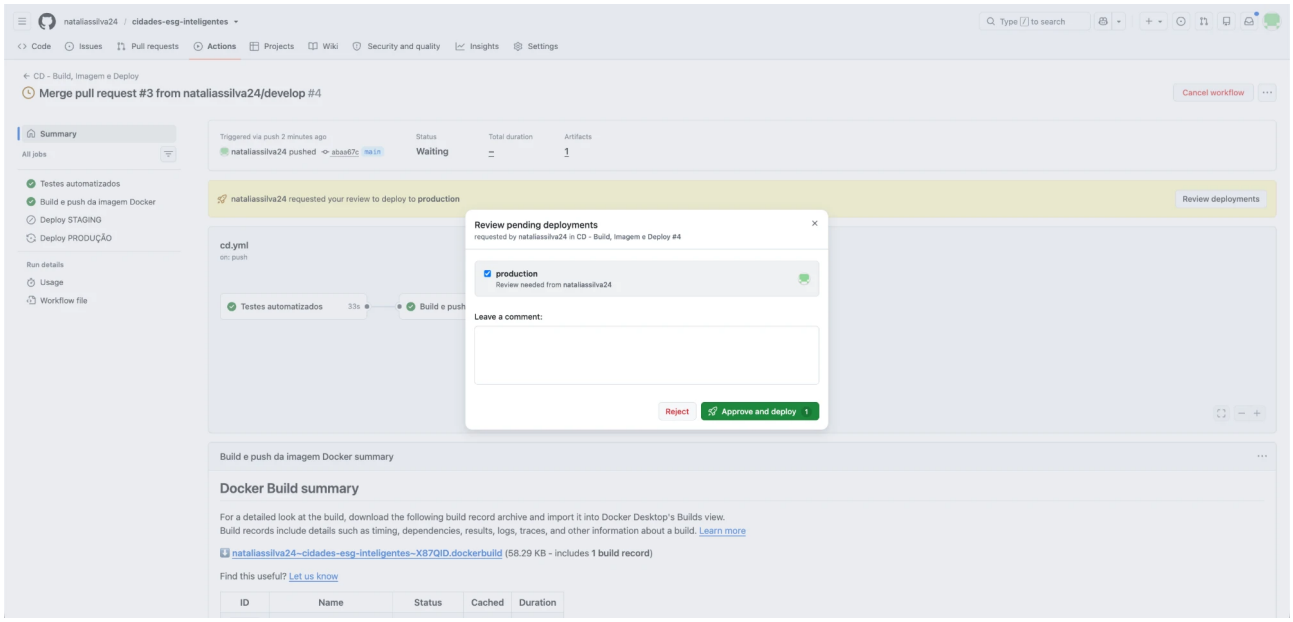


Figura 5 - Revisão do deploy pendente (Review pending deployments)

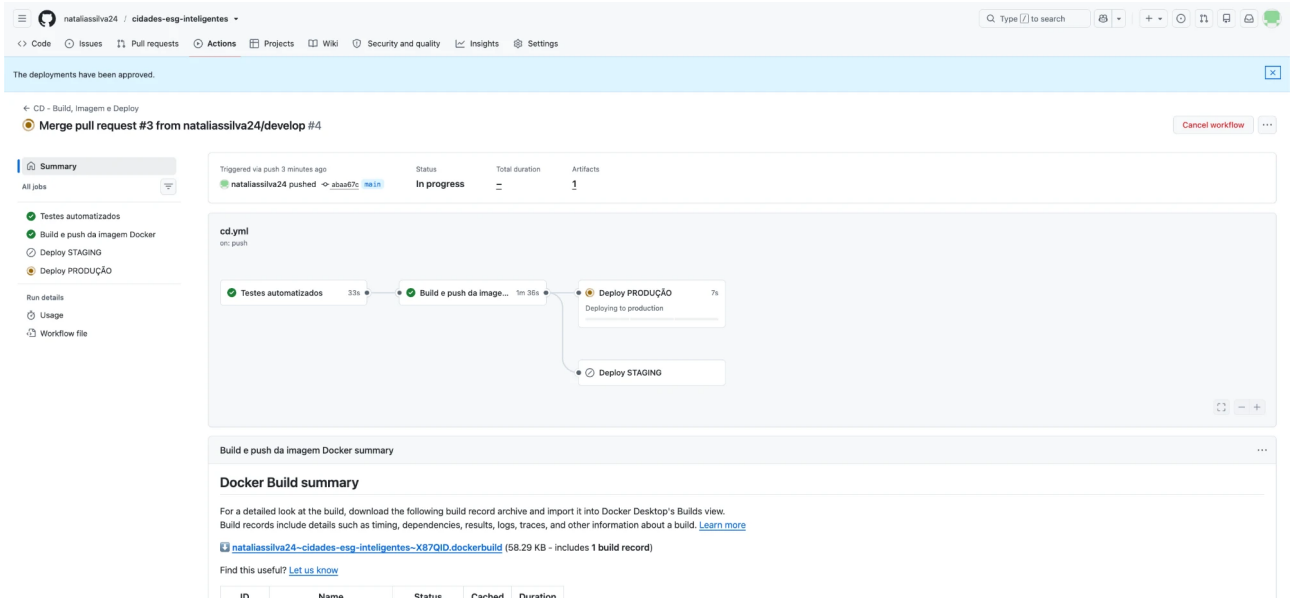


Figura 6 - Deploy aprovado e em execução

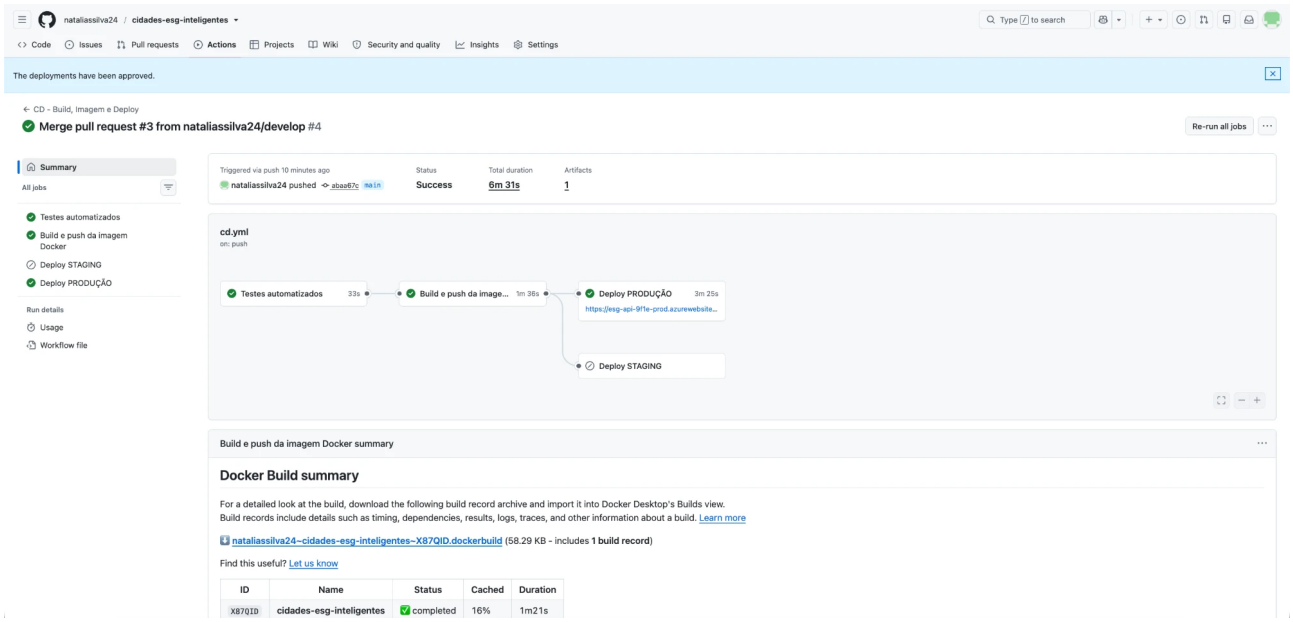


Figura 7 - Pipeline de produção concluído com sucesso, com a URL do ambiente

5. Ambientes funcionando

A rota pública / mostra o ambiente e a versão em execução (SHA do commit), e assim dá para saber qual build está em cada ambiente. Além dos prints, as rotas foram testadas nos dois ambientes: login JWT, permissões por perfil, pacientes, prontuários, exames e o indicador ESG.



Figura 8 - Staging: ambiente 'staging' com a versão 61dd45f (merge do PR #2)

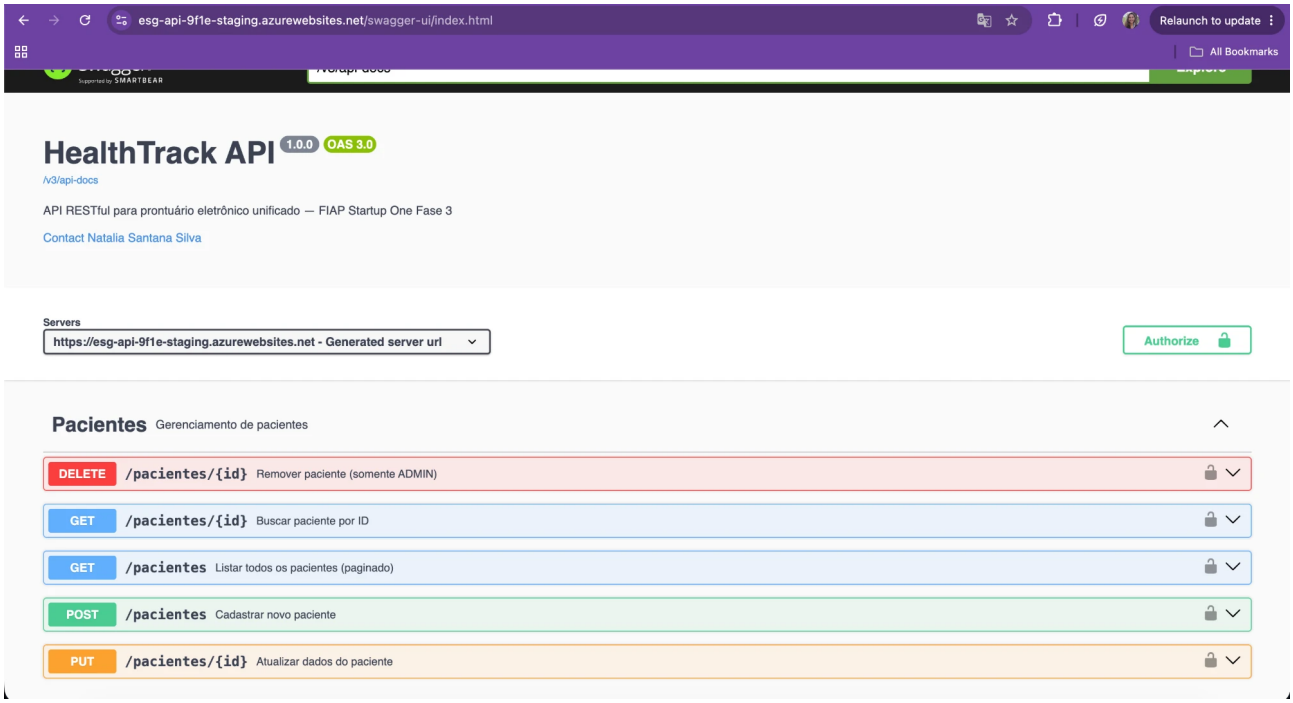


Figura 9 - Staging: documentação Swagger da HealthTrack API



Figura 10 - Produção: ambiente 'production' com a versão abaa67c (merge do PR #3)

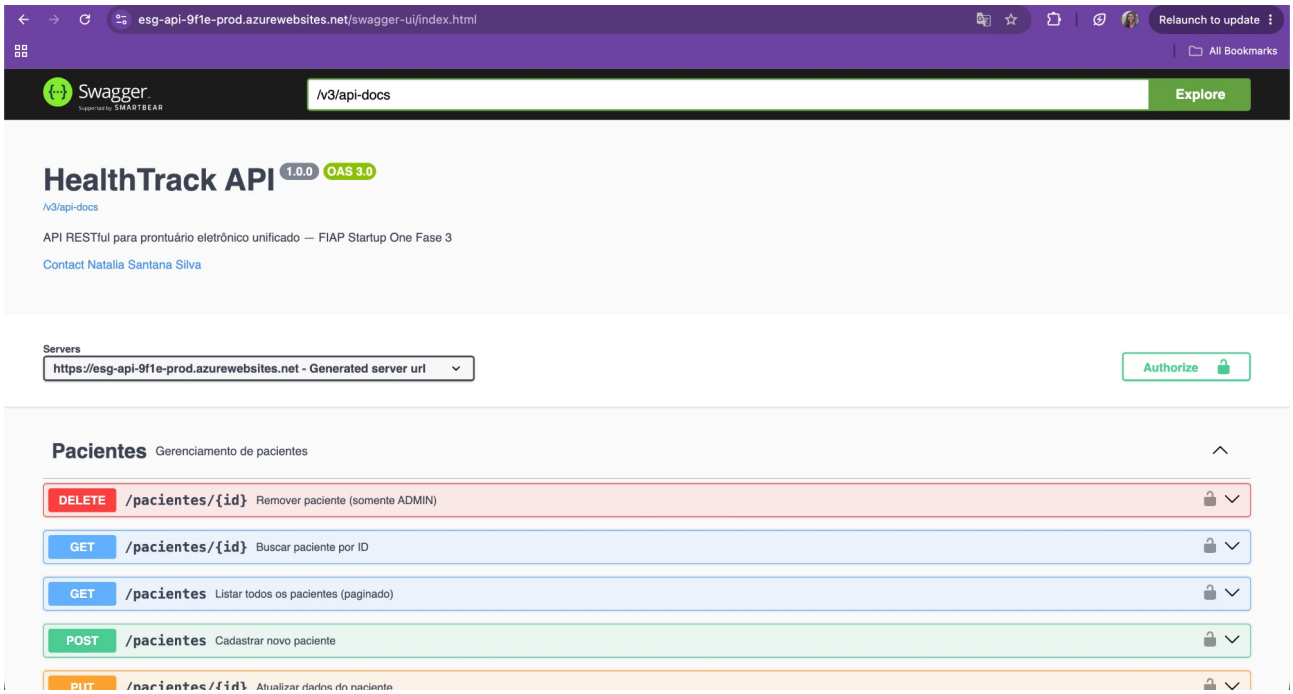


Figura 11 - Produção: documentação Swagger da HealthTrack API

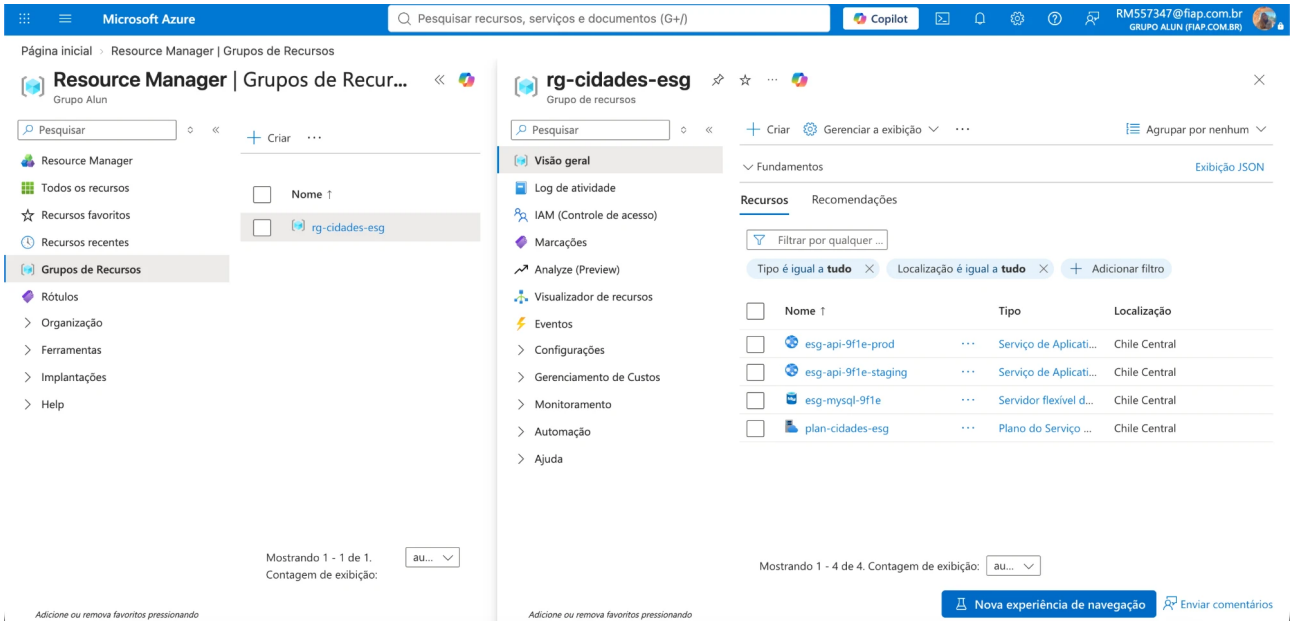


Figura 12 - Recursos no Azure (Resource Group rg-cidades-esg, região Chile Central)

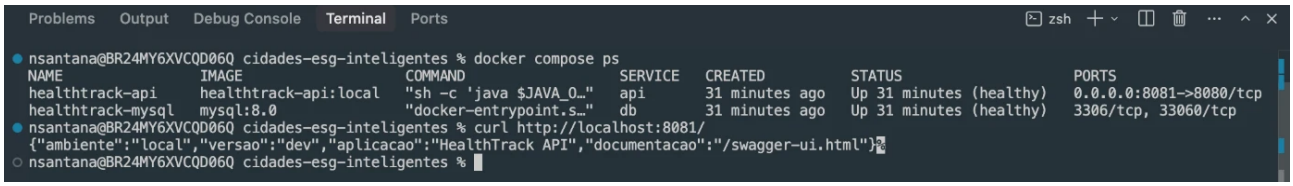


Figura 13 - Ambiente local: API e MySQL healthy via Docker Compose

6. Desafios encontrados e como foram resolvidos

Desafio	Solução
A política da assinatura Azure for Students só permite 5 regiões, sem Brazil South.	Consulta das políticas atribuídas (az policy assignment list) e uso da região permitida mais próxima: Chile Central.
O projeto original usava o Oracle da FIAP, com um único schema: não seria possível ter staging e produção isolados.	Migração para MySQL: migrations reescritas, sequences trocadas por AUTO_INCREMENT e entidades com GenerationType.IDENTITY. Bancos healthtrack_staging e healthtrack_prod separados.
Após a migração, o cadastro retornava erro 500: tabela 'usuarios' inexistente.	O MySQL em Linux diferencia maiúsculas de minúsculas e o Hibernate gera nomes em minúsculas; as migrations passaram a criar as tabelas em minúsculas.
O histórico de exames retornou erro 500 após um ajuste de configuração JPA (open-in-view desativado).	Identificado o carregamento lazy do paciente fora da transação; mantido o comportamento padrão do projeto original.
Os usuários de teste do projeto original não conseguiam autenticar.	Os hashes BCrypt da migration V5 não correspondiam às senhas documentadas; foram regenerados.
Credenciais do banco e chave JWT tinham valores padrão no código.	Configuração 100% por variáveis de ambiente, .env.example versionado, secrets por GitHub Environment e App Settings no Azure, com chave JWT diferente por ambiente.
O Azure CLI falhava em alguns comandos por incompatibilidade do Python 3.14 com a libexpat do macOS.	Execução do CLI com a libexpat atualizada do Homebrew, sem alterar o script versionado.
O script de provisionamento encerrava sem mensagem de erro.	Diagnóstico de SIGPIPE (código 141) com set -o pipefail na geração de valores aleatórios; troca por openssl rand.
Risco de publicar em produção sem validação.	Fluxo develop -> staging e main -> produção, com aprovação manual obrigatória no environment production e smoke test após cada deploy.
Porta 8080 ocupada na máquina local.	Porta publicada configurável pela variável APP_PORT no .env.

7. Tecnologias utilizadas

Camada	Tecnologias
Aplicação	Java 21, Spring Boot 3.2 (Web, Data JPA, Validation, Security, Actuator), JWT (jjwt), Lombok, springdoc-openapi
Banco de dados	MySQL 8, Azure Database for MySQL Flexible Server, Flyway
Testes	JUnit 5, Spring Boot Test, MockMvc, Mockito, Spring Security Test
Build	Maven
Containers	Docker (multi-stage), Docker Compose
CI/CD	GitHub Actions, GitHub Environments, GitHub Container Registry
Nuvem	Microsoft Azure: App Service (Web App for Containers, Linux B1), Azure CLI

8. Checklist de entrega

Item	OK
Projeto compactado em .ZIP com estrutura organizada	Sim
Dockerfile funcional	Sim
docker-compose.yml ou arquivos Kubernetes	Sim
Pipeline com etapas de build, teste e deploy	Sim
README.md com instruções e prints	Sim
Documentação técnica com evidências (PDF ou PPT)	Sim
Deploy realizado nos ambientes staging e produção	Sim